K8S integration with gitlab

Adding and removing Kubernetes clusters

Kubernetes clusters

GitLab offers integrated cluster creation for the following Kubernetes providers:

- Google Kubernetes Engine (GKE).
- Amazon Elastic Kubernetes Service (EKS).

GitLab can also integrate with any standard Kubernetes provider, either on-premise or hosted.

Before you begin

Before adding a Kubernetes cluster using GitLab, you need:

- GitLab itself. Either:
  - A GitLab.com account.
  - A self-managed installation with GitLab version 12.5 or later. This ensures the GitLab UI can be used for cluster creation.
- The following GitLab access:
  - Maintainer access to a project for a project-level cluster.
  - Maintainer access to a group for a group-level cluster.
  - Admin Area access for a self-managed instance-level cluster.

Access controls

When creating a cluster in GitLab, you are asked if you would like to create either:

- A Role-based access control (RBAC) cluster, which is the GitLab default and recommended option.
- An Attribute-based access control (ABAC) cluster.

GitLab creates the necessary service accounts and privileges to install and run GitLab managed applications. When GitLab creates the cluster, a gitlab service account with cluster-admin privileges is created in the default namespace to manage the newly created cluster.

The first time you install an application into your cluster, the tiller service account is created with cluster-admin privileges in the gitlab-managed-apps namespace.

The resources created by GitLab differ depending on the type of cluster.

Important notes

Note the following about access controls:

- Environment-specific resources are only created if your cluster is managed by GitLab.
- If your cluster was created before GitLab 12.2, it uses a single namespace for all project environments.

RBAC cluster resources

GitLab creates the following resources for RBAC clusters.

Name	Type	Details	Created when
`gitlab`	`ServiceAccount`	`default` namespace	Creating a new cluster
`gitlab-admin`	`ClusterRoleBinding`	`cluster-admin` roleRef	Creating a new cluster
`gitlab-token`	`Secret`	Token for `gitlab` ServiceAccount	Creating a new cluster
`tiller`	`ServiceAccount`	`gitlab-managed-apps` namespace	Installing Helm charts
`tiller-admin`	`ClusterRoleBinding`	`cluster-admin` roleRef	Installing Helm charts
Environment namespace	`Namespace`	Contains all environment-specific resources	Deploying to a cluster
Environment namespace	`ServiceAccount`	Uses namespace of environment	Deploying to a cluster
Environment namespace	`Secret`	Token for environment ServiceAccount	Deploying to a cluster
Environment namespace	`RoleBinding`	`admin` roleRef	Deploying to a cluster

ABAC cluster resources

GitLab creates the following resources for ABAC clusters

Name	Type	Details	Created when
`gitlab`	`ServiceAccount`	`default` namespace	Creating a new cluster
`gitlab-token`	`Secret`	Token for `gitlab` ServiceAccount	Creating a new cluster
`tiller`	`ServiceAccount`	`gitlab-managed-apps` namespace	Installing Helm charts
`tiller-admin`	`ClusterRoleBinding`	`cluster-admin` roleRef	Installing Helm charts
Environment namespace	`Namespace`	Contains all environment-specific resources	Deploying to a cluster
Environment namespace	`ServiceAccount`	Uses namespace of environment	Deploying to a cluster
Environment namespace	`Secret`	Token for environment ServiceAccount	Deploying to a cluster

Security of runners

Runners have the privileged mode enabled by default, which allows them to execute special commands and run Docker in Docker. This functionality is needed to run some of the Auto DevOps jobs. This implies the containers are running in privileged mode and you should, therefore, be aware of some important details.

The privileged flag gives all capabilities to the running container, which in turn can do almost everything that the host can do. Be aware of the inherent security risk associated with performing docker run operations on arbitrary images as they effectively have root access.

If you don’t want to use a runner in privileged mode, either:

- Use shared runners on GitLab.com. They don’t have this security issue.
- Set up your own runners using the configuration described at shared runners. This involves:
    1. Making sure that you don’t have it installed via the applications.
    2. Installing a runner using docker+machine.

Create new cluster

New clusters can be created using GitLab on Google Kubernetes Engine (GKE) or Amazon Elastic Kubernetes Service (EKS) at the project, group, or instance level:

1. Navigate to your:
    - Project's Operations > Kubernetes page, for a project-level cluster.
    - Group's Kubernetes page, for a group-level cluster.
    - Admin Area > Kubernetes page, for an instance-level cluster.
2. Click Add Kubernetes cluster.
3. Click the Create new cluster tab.
4. Click either Amazon EKS or Google GKE, and follow the instructions for your desired service:
    - Amazon EKS.
    - Google GKE.

Add existing cluster

If you have an existing Kubernetes cluster, you can add it to a project, group, or instance.

Kubernetes integration isn't supported for arm64 clusters. After adding an existing cluster, you can install runners for it as described in GitLab Managed Apps.

Existing Kubernetes cluster

To add a Kubernetes cluster to your project, group, or instance:

1. Navigate to your:
    a. Project's **Operations > Kubernetes** page, for a project-level cluster.
    b. Group's **Kubernetes** page, for a group-level cluster.
    c. **Admin Area > Kubernetes** page, for an instance-level cluster.

Add existing cluster

2. Click Add Kubernetes cluster.
3. Click the Add existing cluster tab and fill in the details:
    a. Kubernetes cluster name (required) - The name you wish to give the cluster.
    b. Environment scope (required)
    c. API URL (required) - It's the URL that GitLab uses to access the Kubernetes API. Kubernetes exposes several APIs, we want the "base" URL that is common to all of them. For example, https://kubernetes.example.com rather than https://kubernetes.example.com/api/v1.
    Get the API URL by running this command:

```
kubectl cluster-info | grep -E 'Kubernetes master|Kubernetes control plane' | awk '/http/ {print $NF}'
```

    d. CA certificate (required) - A valid Kubernetes certificate is needed to authenticate to the cluster. We use the certificate created by default.
        i. List the secrets with kubectl get secrets, and one should be named similar to default-token-xxxxx. Copy that token name for use below.
        ii. Get the certificate by running this command:

```
kubectl get secret <secret name> -o jsonpath="{['data']['ca\.crt']}" | base64 --decode
```

Add existing cluster

If the command returns the entire certificate chain, you must copy the Root CA certificate and any intermediate certificates at the bottom of the chain. A chain file has following structure:

e. Token - GitLab authenticates against Kubernetes using service tokens, which are scoped to a particular namespace. The token used should belong to a service account with cluster-admin privileges. To create this service account:

i. Create a file called gitlab-admin-service-account.yaml with contents:

```yaml
apiVersion: v1
kind: ServiceAccount
metadata:
  name: gitlab
  namespace: kube-system
---
apiVersion: rbac.authorization.k8s.io/v1beta1
kind: ClusterRoleBinding
metadata:
  name: gitlab-admin
roleRef:
  apiGroup: rbac.authorization.k8s.io
  kind: ClusterRole
  name: cluster-admin
subjects:
  - kind: ServiceAccount
    name: gitlab
    namespace: kube-system
```

Add existing cluster

ii. Apply the service account and cluster role binding to your cluster:

```
kubectl apply -f gitlab-admin-service-account.yaml
```

You need the `container.clusterRoleBindings.create` permission to create cluster-level roles. If you do not have this permission, you can alternatively enable Basic Authentication and then run the `kubectl apply` command as an administrator:

```
kubectl apply -f gitlab-admin-service-account.yaml --username=admin --password=<password>
```

Output:

```
serviceaccount "gitlab" created
clusterrolebinding "gitlab-admin" created
```

Add existing cluster

iii. Retrieve the token for the `gitlab` service account:

```
kubectl -n kube-system describe secret $(kubectl -n kube-system get secret | grep gitlab | awk '{print $1}')
```

Copy the `<authentication_token>` value from the output:

```
Name:         gitlab-token-b5zv4
Namespace:    kube-system
Labels:       <none>
Annotations:  kubernetes.io/service-account.name=gitlab
              kubernetes.io/service-account.uid=bcfe66ac-39be-11e8-97e8-026dce96b6e8

Type:  kubernetes.io/service-account-token

Data
====
ca.crt:     1025 bytes
namespace:  11 bytes
token:      <authentication_token>
```

Add existing cluster

f. GitLab-managed cluster - Leave this checked if you want GitLab to manage namespaces and service accounts for this cluster.

g. Project namespace (optional) - You don't have to fill it in; by leaving it blank, GitLab creates one for you. Also:

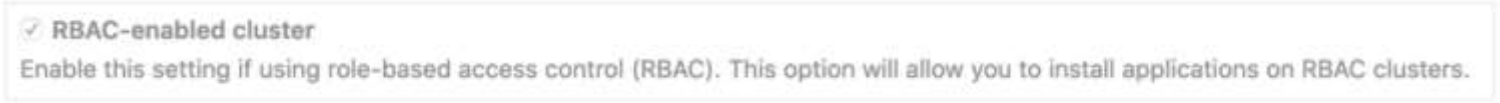
- Each project should have a unique namespace.
- The project namespace is not necessarily the namespace of the secret, if you're using a secret with broader permissions, like the secret from default.
- You should not use default as the project namespace.
- If you or someone created a secret specifically for the project, usually with limited permissions, the secret's namespace and project namespace may be the same.

3. Finally, click the **Create Kubernetes** cluster button.

After a couple of minutes, your cluster is ready. You can now proceed to install some pre-defined applications.

Disable Role-Based Access Control (RBAC) (optional)

When connecting a cluster via GitLab integration, you may specify whether the cluster is RBAC-enabled or not. This affects how GitLab interacts with the cluster for certain operations. If you did not check the RBAC-enabled cluster checkbox at creation time, GitLab assumes RBAC is disabled for your cluster when interacting with it. If so, you must disable RBAC on your cluster for the integration to work properly.



To effectively disable RBAC, global permissions can be applied granting full access:

```
kubectl create clusterrolebinding permissive-binding \
  --clusterrole=cluster-admin \
  --user=admin \
  --user=kubelet \
  --group=system:serviceaccounts
```

Enabling or disabling integration

The Kubernetes cluster integration enables after you have successfully either created a new cluster or added an existing one. To disable Kubernetes cluster integration:

1. Navigate to your:
    - Project's **Operations > Kubernetes** page, for a project-level cluster.
    - Group's **Kubernetes** page, for a group-level cluster.
    - **Admin Area > Kubernetes** page, for an instance-level cluster.
2. Click on the name of the cluster.
3. Click the **GitLab Integration** toggle.
4. Click **Save changes**

Removing integration

To remove the Kubernetes cluster integration from your project, first navigate to the **Advanced Settings** tab of the cluster details page and either:

- Select **Remove integration**, to remove only the Kubernetes integration.
- From GitLab 12.6, select **Remove integration and resources**, to also remove all related GitLab cluster resources (for example, namespaces, roles, and bindings) when removing the integration.

When removing the cluster integration, note:

- You need Maintainer permissions and above to remove a Kubernetes cluster integration.
- When you remove a cluster, you only remove its relationship to GitLab, not the cluster itself. To remove the cluster, you can do so by visiting the GKE or EKS dashboard, or using `kubectl`.

Kubernetes clusters

Setting up

Supported cluster versions

GitLab is committed to support at least two production-ready Kubernetes minor versions at any given time. We regularly review the versions we support, and provide a three-month deprecation period before we remove support of a specific version. The range of supported versions is based on the evaluation of:

- The versions supported by major managed Kubernetes providers.
- The versions supported by the Kubernetes community.

Setting up

GitLab supports the following Kubernetes versions, and you can upgrade your Kubernetes version to any supported version at any time:

- 1.19 (support ends on February 22, 2022)
- 1.18 (support ends on November 22, 2021)
- 1.17 (support ends on September 22, 2021)
- 1.16 (support ends on July 22, 2021)
- 1.15 (support ends on May 22, 2021)
- 1.14 (deprecated, support ends on December 22, 2020)

Some GitLab features may support versions outside the range provided here.

Setting up

Multiple Kubernetes clusters

You can associate more than one Kubernetes cluster to your project. That way you can have different clusters for different environments, like development, staging, production, and so on. Add another cluster, like you did the first time, and make sure to set an environment scope that differentiates the new cluster from the rest.

Setting the environment scope

When adding more than one Kubernetes cluster to your project, you need to differentiate them with an environment scope. The environment scope associates clusters with environments similar to how the environment-specific variables work.

The default environment scope is *, which means all jobs, regardless of their environment, use that cluster. Each scope can be used only by a single cluster in a project, and a validation error occurs if otherwise. Also, jobs that don’t have an environment keyword set can’t access any cluster.

Setting up

For example, let's say the following Kubernetes clusters exist in a project:

Cluster	Environment scope
Development	`*`
Production	`production`

And the following environments are set in `.gitlab-ci.yml`:

```
stages:
  - test
  - deploy

test:
  stage: test
  script: sh test

deploy to staging:
  stage: deploy
  script: make deploy
  environment:
    name: staging
    url: https://staging.example.com/

deploy to production:
  stage: deploy
  script: make deploy
  environment:
    name: production
    url: https://example.com/
```

The results:

- The Development cluster details are available in the deploy to staging job.
- The production cluster details are available in the deploy to production job.
- No cluster details are available in the test job because it doesn't define any environment.

Configuring your Kubernetes cluster

After adding a Kubernetes cluster to GitLab, read this section that covers important considerations for configuring Kubernetes clusters with GitLab.

Security implications

The default cluster configuration grants access to a wide set of functionalities needed to successfully build and deploy a containerized application. Bear in mind that the same credentials are used for all the applications running on the cluster.

GitLab-managed clusters

You can choose to allow GitLab to manage your cluster for you. If your cluster is managed by GitLab, resources for your projects are automatically created. If you choose to manage your own cluster, project-specific resources aren’t created automatically. If you are using Auto DevOps, you must explicitly provide the KUBE_NAMESPACE deployment variable for your deployment jobs to use. Otherwise, a namespace is created for you.

Configuring your Kubernetes cluster

Important notes

Note the following with GitLab and clusters:

- If you install applications on your cluster, GitLab will create the resources required to run these even if you have chosen to manage your own cluster.
- Be aware that manually managing resources that have been created by GitLab, like namespaces and service accounts, can cause unexpected errors. If this occurs, try clearing the cluster cache.

Configuring your Kubernetes cluster

Clearing the cluster cache

Introduced in GitLab 12.6.

If you allow GitLab to manage your cluster, GitLab stores a cached version of the namespaces and service accounts it creates for your projects. If you modify these resources in your cluster manually, this cache can fall out of sync with your cluster. This can cause deployment jobs to fail.

To clear the cache:

1. Navigate to your project's **Operations > Kubernetes** page, and select your cluster.
2. Expand the **Advanced settings** section.
3. Click **Clear cluster cache**.

Base domain

Introduced in GitLab 11.8.

You do not need to specify a base domain on cluster settings when using GitLab Serverless. The domain in that case is specified as part of the Knative installation.

Specifying a base domain automatically sets KUBE_INGRESS_BASE_DOMAIN as an environment variable. If you are using Auto DevOps, this domain is used for the different stages. For example, Auto Review Apps and Auto Deploy.

The domain should have a wildcard DNS configured to the Ingress IP address. After Ingress has been installed , you can either:

- Create an A record that points to the Ingress IP address with your domain provider.
- Enter a wildcard DNS address using a service such as nip.io or xip.io. For example, 192.168.1.1.xip.io.

Base domain

To determine the external Ingress IP address, or external Ingress hostname:

- If the cluster is on GKE:
    1. Click the **Google Kubernetes Engine** link in the **Advanced settings**, or go directly to the Google Kubernetes Engine dashboard.
    2. Select the proper project and cluster.
    3. Click **Connect**
    4. Execute the gcloud command in a local terminal or using the **Cloud Shell**.
- *If the cluster is not on GKE*: Follow the specific instructions for your Kubernetes provider to configure kubectl with the right credentials. The output of the following examples show the external endpoint of your cluster. This information can then be used to set up DNS entries and forwarding rules that allow external access to your deployed applications.

Base domain

Depending an your Ingress, the external IP address can be retrieved in various ways. This list provides a generic solution, and some GitLab-specific approaches:

- In general, you can list the IP addresses of all load balancers by running:

```
kubectl get svc --all-namespaces -o jsonpath='{range.items[?(@.status.loadBalancer.ingress)]}{.status.loadBalancer.ingress[*].ip} '
```

- If you installed Ingress using the **Applications**, run:

```
kubectl get service --namespace=gitlab-managed-apps ingress-nginx-ingress-controller -o jsonpath='{.status.loadBalancer.ingress[0].ip}'
```

- Some Kubernetes clusters return a hostname instead, like Amazon EKS. For these platforms, run:

```
kubectl get service --namespace=gitlab-managed-apps ingress-nginx-ingress-controller -o jsonpath='{.status.loadBalancer.ingress[0].hostname}'
```

Base domain

If you use EKS, an Elastic Load Balancer is also created, which incurs additional AWS costs.

- Istio/Knative uses a different command. Run:

```
kubectl get svc --namespace=istio-system istio-ingressgateway -o jsonpath='{.status.loadBalancer.ingress[0].ip} '
```

If you see a trailing `%` on some Kubernetes versions, do not include it.

Installing applications

GitLab can install and manage some applications like Helm, GitLab Runner, Ingress, Prometheus, and so on, in your project-level cluster.

Auto DevOps

Auto DevOps automatically detects, builds, tests, deploys, and monitors your applications.

To make full use of Auto DevOps (Auto Deploy, Auto Review Apps, and Auto Monitoring) the Kubernetes project integration must be enabled. However, Kubernetes clusters can be used without Auto DevOps.

Deploying to a Kubernetes cluster

A Kubernetes cluster can be the destination for a deployment job. If

- The cluster is integrated with GitLab, special deployment variables are made available to your job and configuration is not required. You can immediately begin interacting with the cluster from your jobs using tools such as kubectl or helm.
- You don’t use the GitLab cluster integration, you can still deploy to your cluster. However, you must configure Kubernetes tools yourself using environment variables before you can interact with the cluster from your jobs.

Deployment variables

Deployment variables require a valid Deploy Token named `gitlab-deploy-token`, and the following command in your deployment job script, for Kubernetes to access the registry:

- Using Kubernetes 1.18+:

```
kubectl create secret docker-registry gitlab-registry --docker-server="$CI_REGISTRY" --docker-username="$CI_DEPLOY_USER" --docker-password="$CI_DEPLOY_PASSWORD" --docker-email="$GITLAB_USER_EMAIL" -o yaml --dry-run=client | kubectl apply -f -
```

- **Using Kubernetes <1.18:**

```
kubectl create secret docker-registry gitlab-registry --docker-server="$CI_REGISTRY" --docker-username="$CI_DEPLOY_USER" --docker-password="$CI_DEPLOY_PASSWORD" --docker-email="$GITLAB_USER_EMAIL" -o yaml --dry-run | kubectl apply -f -
```

Deployment variables

The Kubernetes cluster integration exposes these deployment variables in the GitLab CI/CD build environment to deployment jobs. Deployment jobs have defined a target environment.

Variable	Description
`KUBE_URL`	Equal to the API URL.
`KUBE_TOKEN`	The Kubernetes token of the environment service account. Prior to GitLab 11.5, `KUBE_TOKEN` was the Kubernetes token of the main service account of the cluster integration.
`KUBE_NAMESPACE`	The namespace associated with the project's deployment service account. In the format `<project_name>-<project_id>-<environment>`. For GitLab-managed clusters, a matching namespace is automatically created by GitLab in the cluster. If your cluster was created before GitLab 12.2, the default `KUBE_NAMESPACE` is set to `<project_name>-<project_id>`.
`KUBE_CA_PEM_FILE`	Path to a file containing PEM data. Only present if a custom CA bundle was specified.
`KUBE_CA_PEM`	(**deprecated**) Raw PEM data. Only if a custom CA bundle was specified.
`KUBECONFIG`	Path to a file containing `kubeconfig` for this deployment. CA bundle would be embedded if specified. This configuration also embeds the same token defined in `KUBE_TOKEN` so you likely need only this variable. This variable name is also automatically picked up by `kubectl` so you don't need to reference it explicitly if using `kubectl`.
`KUBE_INGRESS_BASE_DOMAIN`	From GitLab 11.8, this variable can be used to set a domain per cluster. See cluster domains for more information.

Custom namespace

The Kubernetes integration provides a `KUBECONFIG` with an auto-generated namespace to deployment jobs. It defaults to using project-environment specific namespaces of the form `<prefix>-<environment>`, where `<prefix>` is of the form `<project_name>-<project_id>`.

You can customize the deployment namespace in a few ways:

- You can choose between a **namespace per environment** or a **namespace per project**. A namespace per environment is the default and recommended setting, as it prevents the mixing of resources between production and non-production environments.
- When using a project-level cluster, you can additionally customize the namespace prefix. When using namespace-per-environment, the deployment namespace is `<prefix>-<environment>`, but otherwise just `<prefix>`.
- For **non-managed** clusters, the auto-generated namespace is set in the `KUBECONFIG`, but the user is responsible for ensuring its existence. You can fully customize this value using `environment:kubernetes:namespace` in `.gitlab-ci.yml`.

Integrations

Canary Deployments

Leverage Kubernetes' Canary deployments and visualize your canary deployments right inside the Deploy Board, without the need to leave GitLab.

Deploy Boards

GitLab Deploy Boards offer a consolidated view of the current health and status of each CI environment running on Kubernetes. They display the status of the pods in the deployment. Developers and other teammates can view the progress and status of a rollout, pod by pod, in the workflow they already use without any need to access Kubernetes.

Viewing pod logs

GitLab enables you to view the logs of running pods in connected Kubernetes clusters. By displaying the logs directly in GitLab, developers can avoid having to manage console tools or jump to a different interface.

Integrations

Web terminals

Introduced in GitLab 8.15.

When enabled, the Kubernetes integration adds web terminal support to your environments. This is based on the exec functionality found in Docker and Kubernetes, so you get a new shell session in your existing containers. To use this integration, you should deploy to Kubernetes using the deployment variables above, ensuring any deployments, replica sets, and pods are annotated with:

- `app.gitlab.com/env: $CI_ENVIRONMENT_SLUG`
- `app.gitlab.com/app: $CI_PROJECT_PATH_SLUG`

`$CI_ENVIRONMENT_SLUG` and `$CI_PROJECT_PATH_SLUG` are the values of the CI variables.

You must be the project owner or have `maintainer` permissions to use terminals. Support is limited to the first container in the first pod of your environment.

Troubleshooting

Before the deployment jobs starts, GitLab creates the following specifically for the deployment job:

- A namespace.
- A service account.

However, sometimes GitLab can not create them. In such instances, your job can fail with the message:

```
This job failed because the necessary resources were not successfully created.
```

To find the cause of this error when creating a namespace and service account, check the **logs**.

Reasons for failure include:

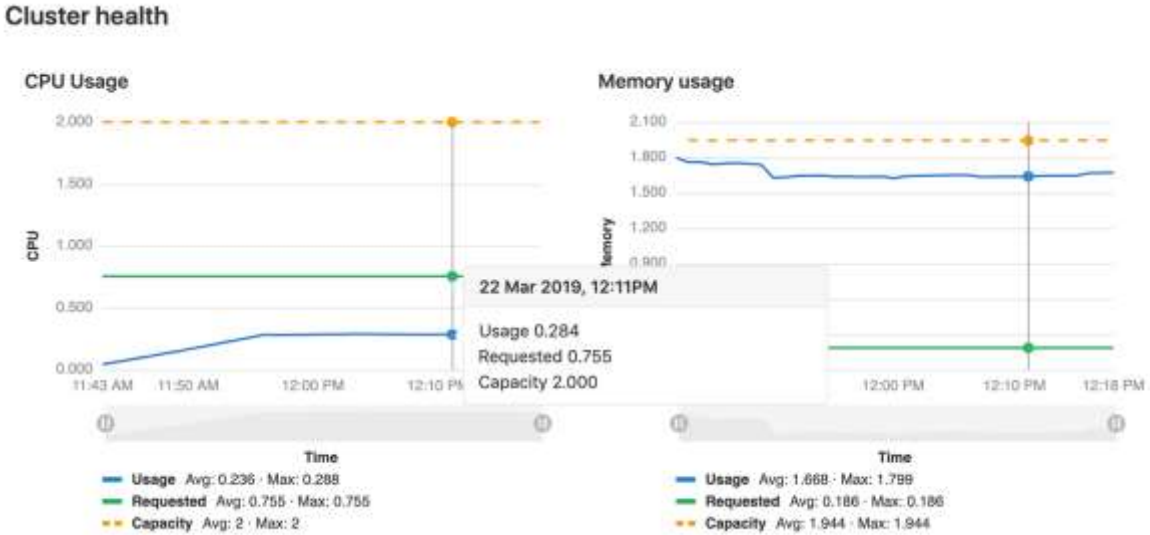
- The token you gave GitLab does not have `cluster-admin` privileges required by GitLab.
- Missing `KUBECONFIG` or `KUBE_TOKEN` variables. To be passed to your job, they must have a matching `environment:name`. If your job has no `environment:name` set, the Kubernetes credentials are not passed to it.

Monitoring your Kubernetes cluster

Automatically detect and monitor Kubernetes metrics. Automatic monitoring of NGINX Ingress is also supported.

Visualizing cluster health

When Prometheus is deployed, GitLab monitors the cluster's health. At the top of the cluster settings page, CPU and Memory utilization is displayed, along with the total amount available. Keeping an eye on cluster resources can be important, if the cluster runs out of memory pods may be shutdown or fail to start.



Q&A